Digital Image Processing

# Lecture 4: Spatial Filtering

**Dr. Behnam Kiani**
**School of Applied Intelligence**

**Spring 2026**

Readings: Chapter 3 (sections 3.5-3.8), Chapter 4 (section 4.1)
How to read: Phones OFF! Pens ON!

# Table of Content


- Smoothing (Lowpass) Spatial Filters
- Sharpening (Highpass) Spatial Filters
- Highpass, Bandreject and Bandpass Filters From Lowpass Filters
- Combining Spatial Enhancement Methods

# Smoothing Spatial Filters

- Smoothing (averaging) spatial filters are used for reducing sharp transitions in intensity.
- Random noise consists sharp transitions; therefore they reduce noise too.
- Smoothing prior to re-sampling reduces aliasing (Chapter 4).
- Smoothing can reduce false contouring.
- Two types:
  - Linear filters:
    - Box Filter Kernels
    - Lowpass Gaussian Filter Kernels
  - Nonlinear filters:
    - Order-Statistic Filters

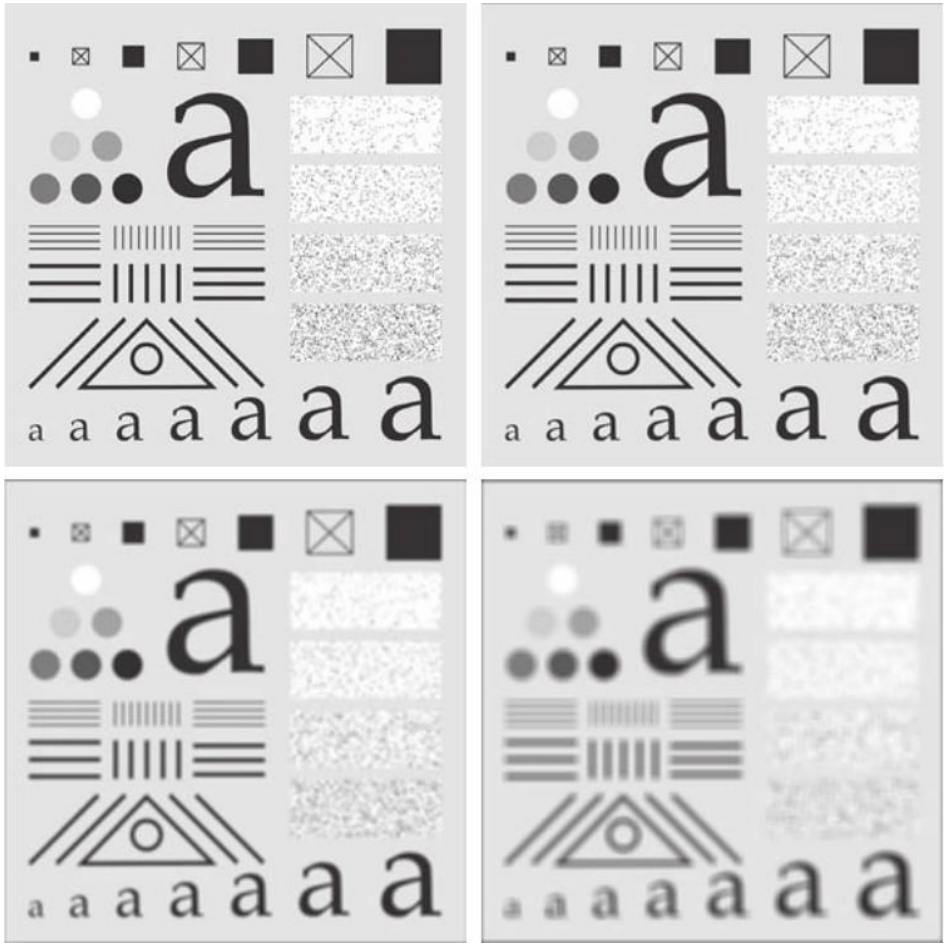
# Box Filter Kernels

- Simple, separable lowpass filter kernel, with coefficients of identical values in a small neighborhood.
- An **mxn** box filter, a matrix of **1**'s, with a normalizing constant equal to the number of elements in the matrix.
- Why normalize the kernel?
  - An area of constant intensity will remain the same after filtering;
  - Preventing bias during filtering, sum of the pixels in the original and filtered images are the same.

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

# Box Filter Kernels: Example

- Original image: **1024x1024** pixels.
- Kernels of size **3x3**, **11x11** and **21x21**.
- Gray border as a result of zero-padding the image to avoid undefined operations when part of the kernel is outside the image.



# Lowpass Gaussian Filter Kernels

- Box filters are simple and suitable for quick implementations. Useful when we want to reduce the effect of smoothing on edges.
- Box filters are poor approximations of blurring characteristics of lenses.
- Box filters favor blurring along the perpendicular lines.
- To avoid the shortcomings, circularly symmetric (isotropic) kernels are preferred.
- Gaussian kernels are circularly symmetric and separable:

$$w(s,t) = G(s,t) = Ke^{-\frac{s^2+t^2}{2\sigma^2}}$$

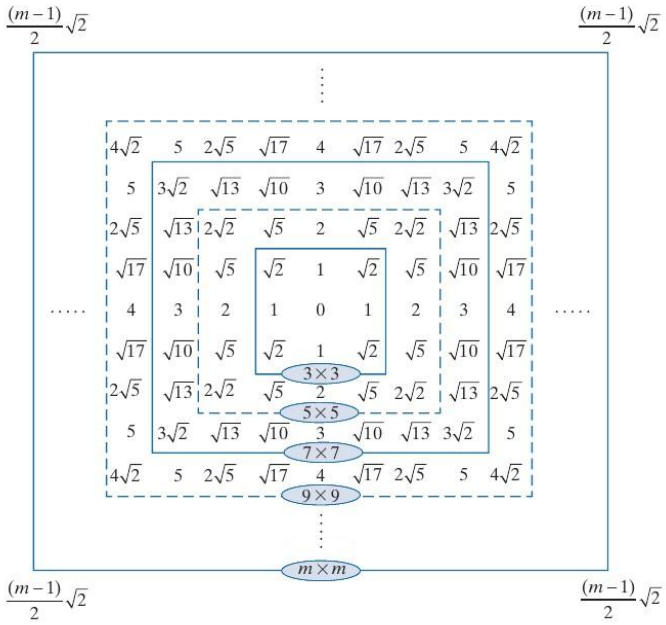
# Lowpass Gaussian Filter Kernels

- To signify the circularly symmetric form of the kernel:

$$w(s,t) = G(s,t) = Ke^{-\frac{s^2+t^2}{2\sigma^2}}$$

$$r = \sqrt{(s^2+t^2)} \longrightarrow w(s,t) = G(s,t) = Ke^{-\frac{r^2}{2\sigma^2}}$$

- Figure shows values of **r** for several kernel sizes:

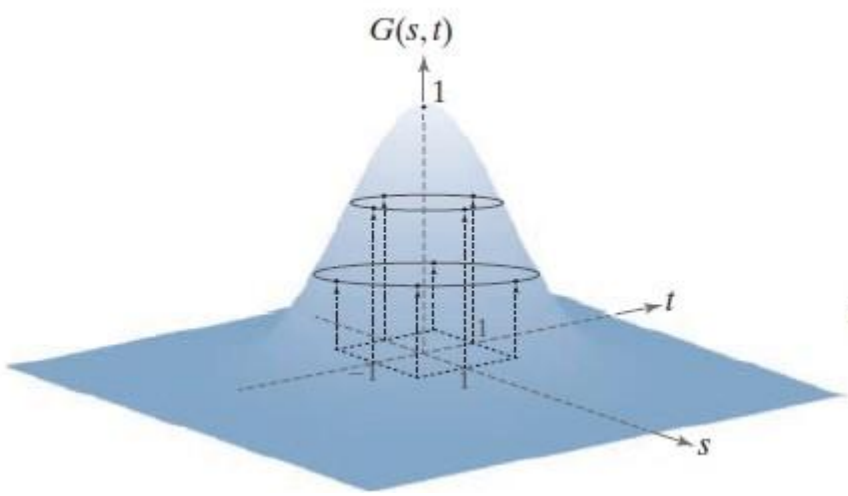


# Lowpass Gaussian Filter Kernels

- The Gaussian function is a continuous function, and cannot be applied directly to the images.
- We need to sample it at integer spatial locations, and make a discrete kernel.
- Normalizing the kernel by dividing its coefficients by the sum of the coefficients completes the process of specifying the kernel.

a b

**FIGURE 3.35** (a) Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for $K = 1$ and $\sigma = 1$. (b) Resulting $3 \times 3$ kernel [this is the same as Fig. 3.31(b)].




$$\frac{1}{4.8976} \times \begin{bmatrix} 0.3679 & 0.6065 & 0.3679 \\ 0.6065 & 1.0000 & 0.6065 \\ 0.3679 & 0.6065 & 0.3679 \end{bmatrix}$$

# Gaussian Kernels Properties

- Since Gaussian kernels are circularly symmetric and separable, sampling can be done only along a cross section through the center, then convert it to a 2D kernel by the procedure explained in previous lecture.
- Values of a Gaussian function at distance larger than **3σ** from the mean are very small, so we can select the size of Gaussian kernel to be:

$$\boxed{\text{Gaussian kernel size} = \lceil 6\sigma \rceil \times \lceil 6\sigma \rceil}$$

with ⌜ **c** ⌝ to denote the ceiling of **c**.

- The product and convolution of two Gaussian kernels are also Gaussian.

| | $f$ | $g$ | $f \times g$ | $f \star g$ |
|---|---|---|---|---|
| Mean | $m_f$ | $m_g$ | $m_{f\times g} = \dfrac{m_f\sigma_g^2 + m_g\sigma_f^2}{\sigma_f^2 + \sigma_g^2}$ | $m_{f\star g} = m_f + m_g$ |
| Standard deviation | $\sigma_f$ | $\sigma_g$ | $\sigma_{f\times g} = \sqrt{\dfrac{\sigma_f^2\sigma_g^2}{\sigma_f^2 + \sigma_g^2}}$ | $\sigma_{f\star g} = \sqrt{\sigma_f^2 + \sigma_g^2}$ |

# Gaussian Kernels: Example

a b c

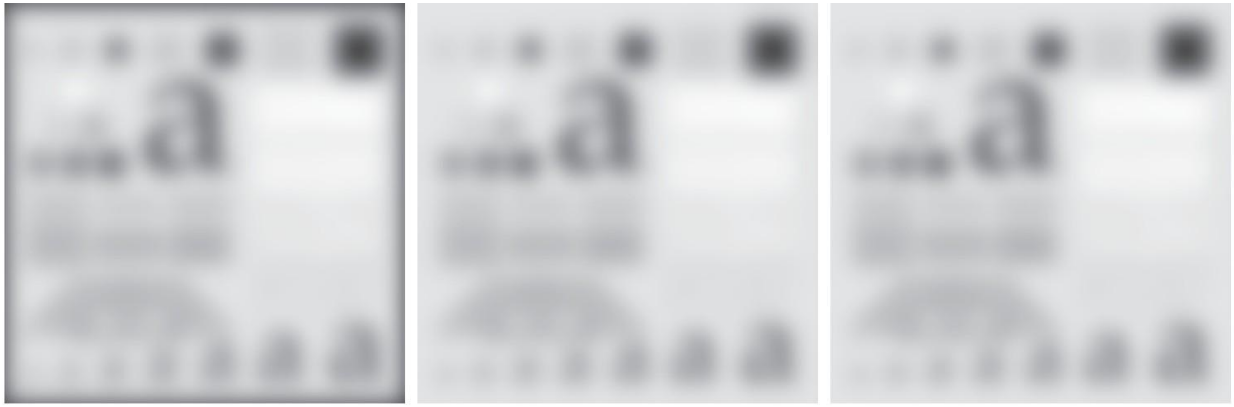
**FIGURE 3.36** (a)A test pattern of size $1024 \times 1024$. (b) Result of lowpass filtering the pattern with a Gaussian kernel of size $21 \times 21$, with standard deviations $\sigma = 3.5$. (c) Result of using a kernel of size $43 \times 43$, with $\sigma = 7$. This result is comparable to Fig. 3.33(d). We used $K = 1$ in all cases.

a b c

**FIGURE 3.37** (a) Result of filtering Fig. 3.36(a) using a Gaussian kernels of size $43 \times 43$, with $\sigma = 7$. (b) Result of using a kernel of $85 \times 85$, with the same value of $\sigma$. (c) Difference image.

# Image Padding?

- We saw image zero-padding when computing correlations and convolutions.
- Padding methods:
  - Zero-padding
  - Mirror (or symmetric) padding: values outside the boundary are obtained by mirror-reflecting the image across the border.
  - Replicate padding: values outside the boundary are set equal to the nearest image border value.



a b c

**FIGURE 3.39** Result of filtering the test pattern in Fig. 3.36(a) using (a) zero padding, (b) mirror padding, and (c) replicate padding. A Gaussian kernel of size $187 \times 187$, with $K = 1$ and $\sigma = 31$ was used in all three cases.

# Example: Shading Correction Using Lowpass Filtering

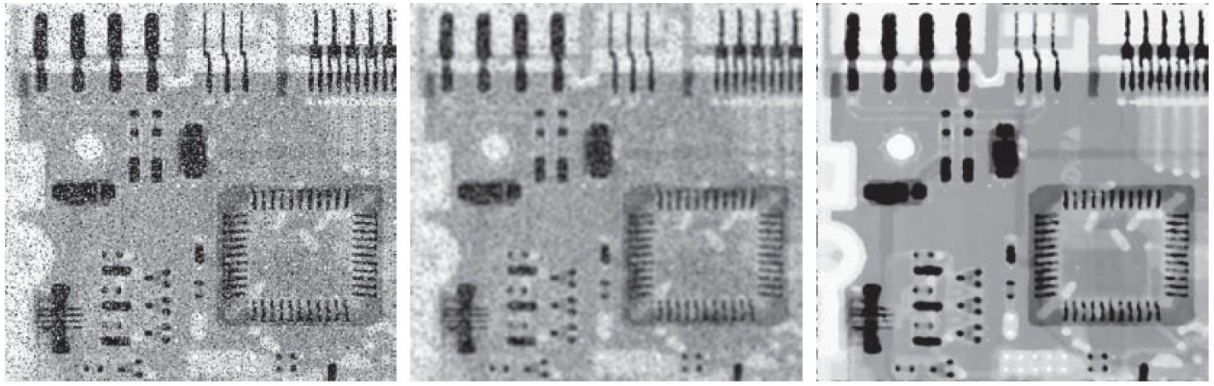
- Shading caused by nonuniform illumination, results in erroneous measurements and degraded performance of automated DIP.
- Shading correction (flat-field correction) can be done by estimating the shading of an image using lowpass filtering.
- Left is a **2048x2048** checkerboard image, with squares of size **128x128**. Middle is the shading estimation resulted from applying Gaussian kernel of size **512x512**, with **K=1** and **σ=128**. Right is the shading corrected image.

a b c

**FIGURE 3.42** (a) Image shaded by a shading pattern oriented in the −45° direction. (b) Estimate of the shading patterns obtained using lowpass filtering. (c) Result of dividing (a) by (b). (See Section 9.8 for a morphological approach to shading correction).

# Order-Statistic (Nonlinear) Filters

- Order-statistic filters are nonlinear filters that work based on ordering (ranking) of the pixels in the region encompassed by the neighborhood.
- **Median Filter** replaces the value of the center pixel by the median of the intensity values defined in the neighborhood of that pixel. Very useful for reducing salt-and-pepper noise.
- Median of a set of values is such that half of the values in the set are less than or equal to the median. When several values are the same, they will be grouped.



a b c

**FIGURE 3.43** (a) X-ray image of a circuit board, corrupted by salt-and-pepper noise. (b) Noise reduction using a $19 \times 19$ Gaussian lowpass filter kernel with $\sigma = 3$. (c) Noise reduction using a $7 \times 7$ median filter. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)

# Sharpening (Highpass) Spatial Filters

- Sharpening highlights transitions in intensity.
- Averaging (smoothing) is analogous to integration, then sharpening is analogous to spatial differentiation.
- Image differentiation enhances edges and other discontinuities (like noise) and de-emphasizes areas of slowly varying intensities.
- First-order derivative:
  - Must be ***zero*** in areas of **constant** intensity;
  - Must be ***non-zero*** at the onset of an intensity **step** or **ramp**;
  - Must be ***non-zero*** along intensity **ramps**.
- Second-order derivative:
  - Must be ***zero*** in areas of **constant** intensity;
  - Must be ***non-zero*** at the onset and end of an intensity **step** or **ramp**;
  - Must be ***zero*** along intensity **ramps**.

# Sharpening (Highpass) Spatial Filters

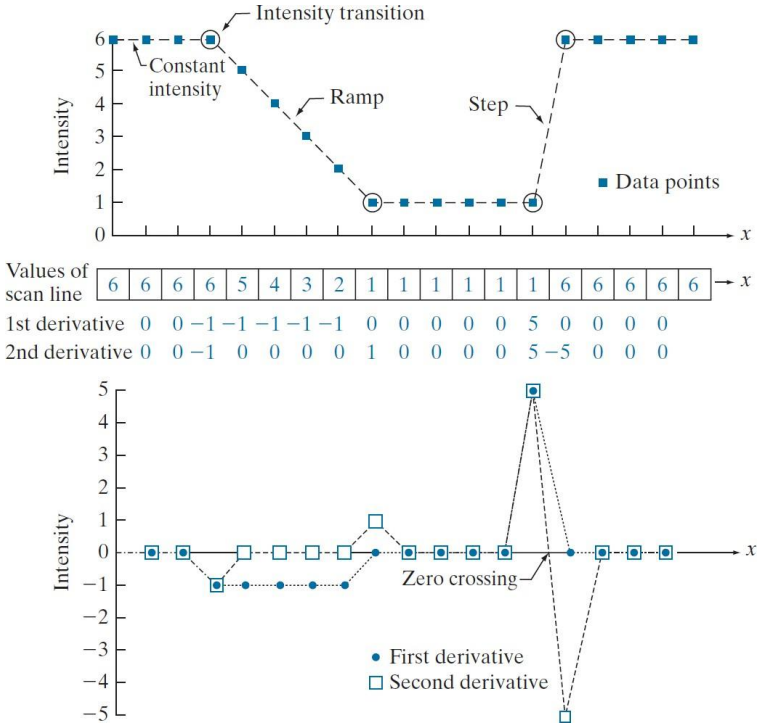
- For digital functions, the derivatives are defined in terms of differences.
- First-order derivative of a 1D function in difference form:

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

- Second-order derivative of a 1D function in difference form:

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$

- Partial derivative is used, so the notation is consistent between 1D and 2D functions.



# Second Order Derivatives for Image Sharpening – The Laplacian

- We are interested in isotropic kernels with response independent of the direction of the intensity discontinuity.
- The Laplacian of a function with two variables:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

- By replacing the two second-derivative terms we have:

$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

| | | |
|---|---|---|
| 0 | 1 | 0 |
| 1 | −4 | 1 |
| 0 | 1 | 0 |

# Second Order Derivatives for Image Sharpening – The Laplacian

- The kernel is isotropic for rotations of increments of 90 degrees.
- What if we want to incorporate the diagonal directions?
- What if the negative sign is used for second derivatives? This needs to be considered when combining a Laplacian-filtered image with another image.
- Note that the coefficients sum to zero, so we don't introduce any bias in the constant regions.

| | | |
|---|---|---|
| 0 | 1 | 0 |
| 1 | −4 | 1 |
| 0 | 1 | 0 |

| | | |
|---|---|---|
| 1 | 1 | 1 |
| 1 | −8 | 1 |
| 1 | 1 | 1 |

| | | |
|---|---|---|
| 0 | −1 | 0 |
| −1 | 4 | −1 |
| 0 | −1 | 0 |

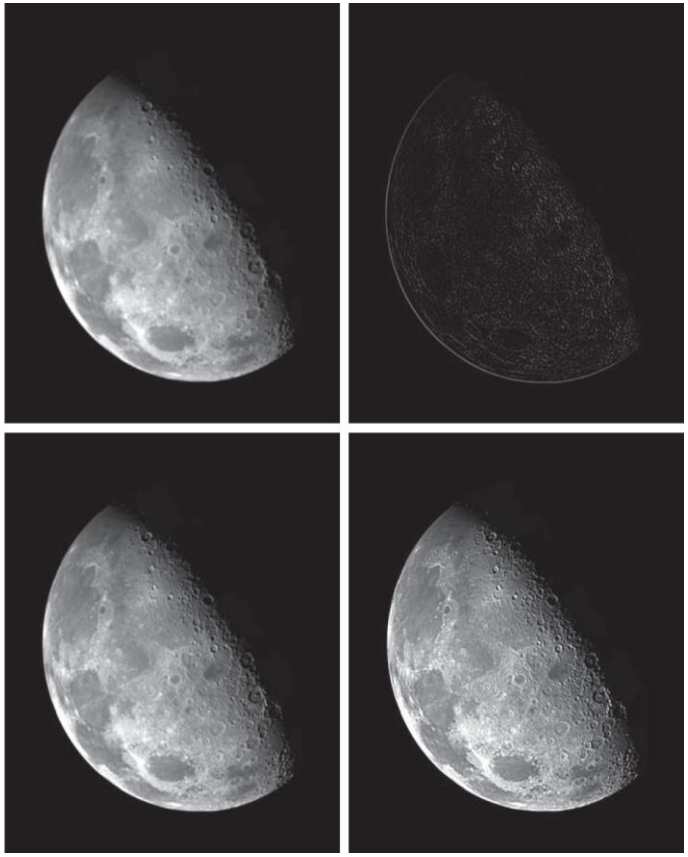
| | | |
|---|---|---|
| −1 | −1 | −1 |
| −1 | 8 | −1 |
| −1 | −1 | −1 |

# Second Order Derivatives for Image Sharpening – The Laplacian

- Now that we derived the Laplacian of an image, we can combine it with the original image to obtain a sharpened version:

$$g(x,y)=f(x,y)+c[\nabla^2 f(x,y)]$$

where **f(x,y)** and **g(x,y)** are the input and sharpened images respectively. If the first two kernels is used, then **c=-1**; Otherwise, **c=1**.



a b
c d

**FIGURE 3.46** (a) Blurred image of the North Pole of the moon. (b) Laplacian image obtained using the kernel in Fig. 3.45(a). (c) Image sharpened using Eq. (3-54) with $c = -1$. (d) Image sharpened using the same procedure, but with the kernel in Fig. 3.45(b). (Original image courtesy of NASA.)

# First Order Derivatives for Image Sharpening – The Gradient

- First derivatives are implemented using the magnitude of the gradient.
- The gradient of an image **f** at coordinates **(x,y)** is defined as a two-dimensional column vector:

$$\nabla f = grad(f) = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

- The vector points at the direction of the greatest rate of change of function **f** at location **(x,y)**.
- The magnitude of the gradient vector, denoted by **M(x,y)** is defined as:

$$\boxed{M(x,y) = \|\nabla f\| = mag(\nabla f) = \sqrt{(g_x^2 + g_y^2)}}$$

which is the value of the rate of change in the direction of the gradient vector at location **(x,y)**.

# First Order Derivatives for Image Sharpening – The Gradient

- A more computationally suitable approximation is done by using:

$$M(x, y) \approx |g_x| + |g_y|$$

- Similar to Laplacian, we can use derivatives in their difference form to create appropriate kernels for computing the gradients:
  - Roberts kernels (cross gradient)
  - Sobel kernels

a
b c
d e

**FIGURE 3.50** (a) A $3 \times 3$ region of an image, where the $z$s are intensity values. (b)–(c) Roberts cross-gradient operators. (d)–(e) Sobel operators. All the kernel coefficients sum to zero, as expected of a derivative operator.

| | | |
|---|---|---|
| $z_1$ | $z_2$ | $z_3$ |
| $z_4$ | $z_5$ | $z_6$ |
| $z_7$ | $z_8$ | $z_9$ |

| | |
|---|---|
| −1 | 0 |
| 0 | 1 |

| | |
|---|---|
| 0 | −1 |
| 1 | 0 |

| | | |
|---|---|---|
| −1 | −2 | −1 |
| 0 | 0 | 0 |
| 1 | 2 | 1 |

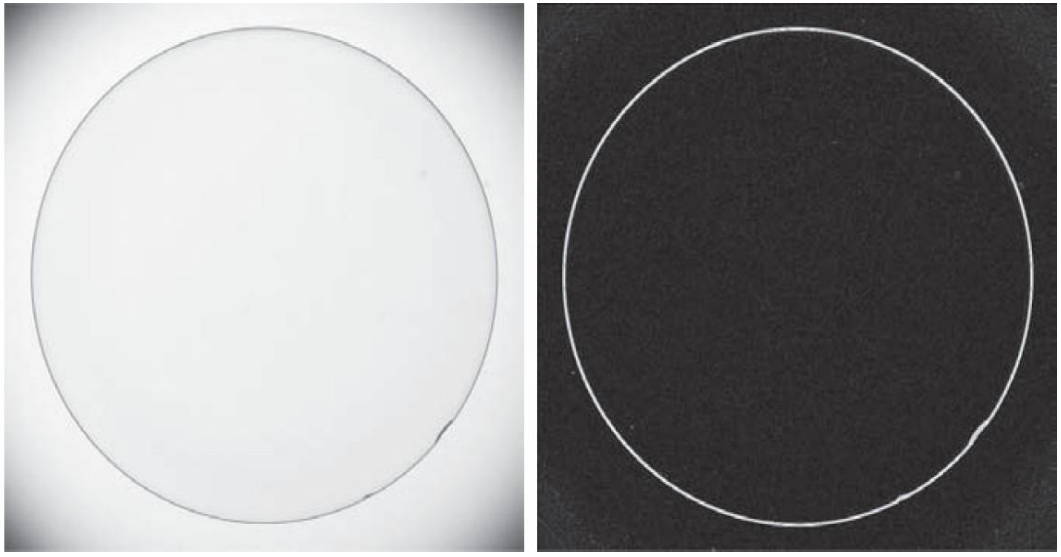
| | | |
|---|---|---|
| −1 | 0 | 1 |
| −2 | 0 | 2 |
| −1 | 0 | 1 |

# First Order Derivatives for Image Sharpening – The Gradient: Example

- Example: detection of defects
  - Converting the original image into the gradient image makes the defects more clear and easier to detect automatically

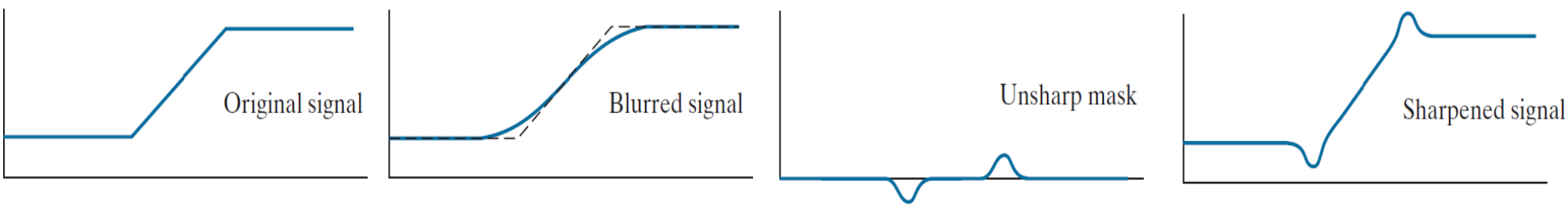
a b

**FIGURE 3.51**
(a) Image of a contact lens (note defects on the boundary at 4 and 5 o'clock).
(b) Sobel gradient.
(Original image courtesy of Perceptics Corporation.)



# Unsharp Masking

- We can subtract an *unsharp* (smoothed) version of an image from the original image to create a mask to sharpen the image.
- The procedure:
  - Blur the original image: $f(x,y) \longrightarrow \bar{f}(x,y)$
  - Subtract the blurred image from the original to create the unsharp mask: $g_{mask}(x,y) = f(x,y) - \bar{f}(x,y)$
  - Add the mask to the original image: $g(x,y) = f(x,y) + k\, g_{mask}(x,y)$



# Unsharp Masking

- When **k=1**, we have unsharp masking. For **k>1**, the process is called highboost filtering.
- Since we may have negative values, clipping might happen, causing dark halos around the edges.

a b c
d e

**FIGURE 3.49** (a) Original image of size $600 \times 259$ pixels. (b) Image blurred using a $31 \times 31$ Gaussian lowpass filter with $\sigma = 5$. (c) Mask. (d) Result of unsharp masking using Eq. (3-56) with $k = 1$. (e) Result of highboost filtering with $k = 4.5$.

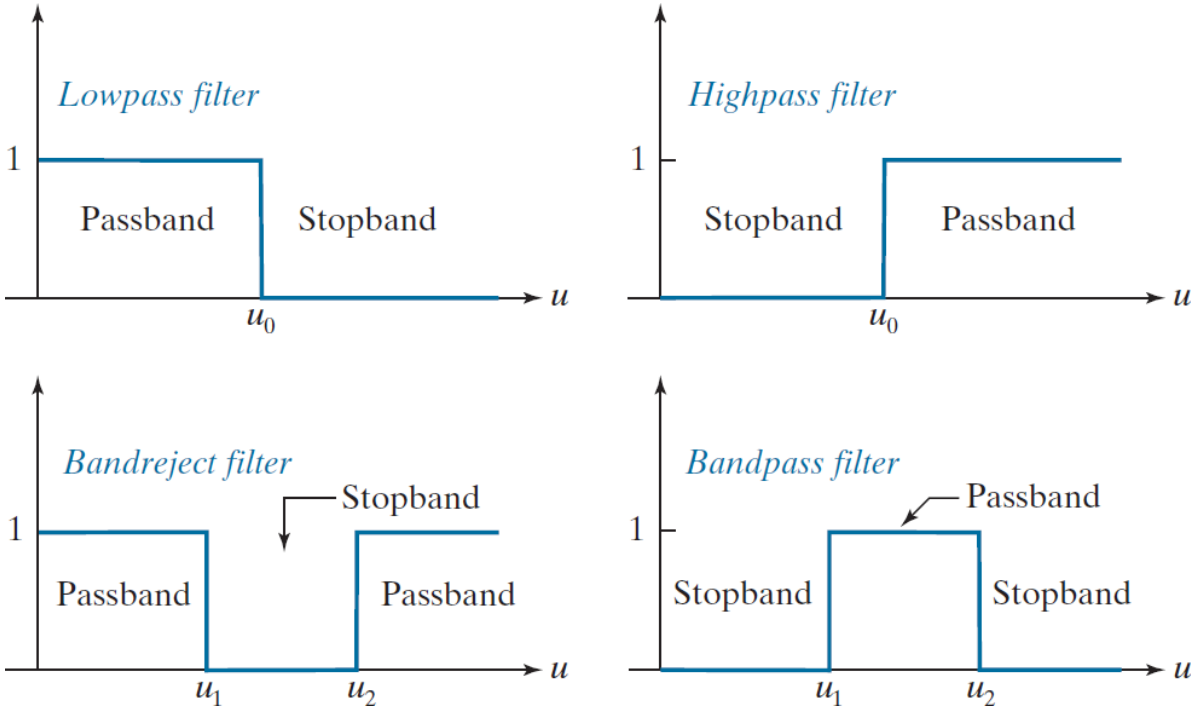
# Highpass, Bandreject, And Bandpass Filters From Lowpass Filters

- Spatial and frequency-domain linear filters are classified into four broad categories: ***lowpass***, ***highpass***, ***bandpass*** and ***bandreject*** filters.
- The last three types can be constructed from lowpass filters.
- Fourier Transform (FT) is used to go from the spatial to frequency domain.
- FT decomposes the signal (image) into its consisting frequency components.
- Two fundamental properties:
  - Convolution in the spatial domain, is equivalent to multiplication in the frequency domain, and vice versa.
  - An **impulse function** of amplitude **A** in the spatial domain, is a constant function of value **A** in the frequency domain, and vice versa.

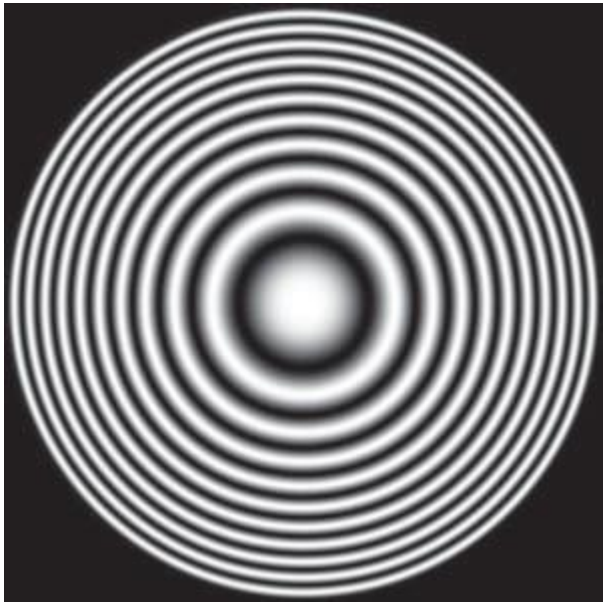
# Highpass, Bandreject, And Bandpass Filters From Lowpass Filters

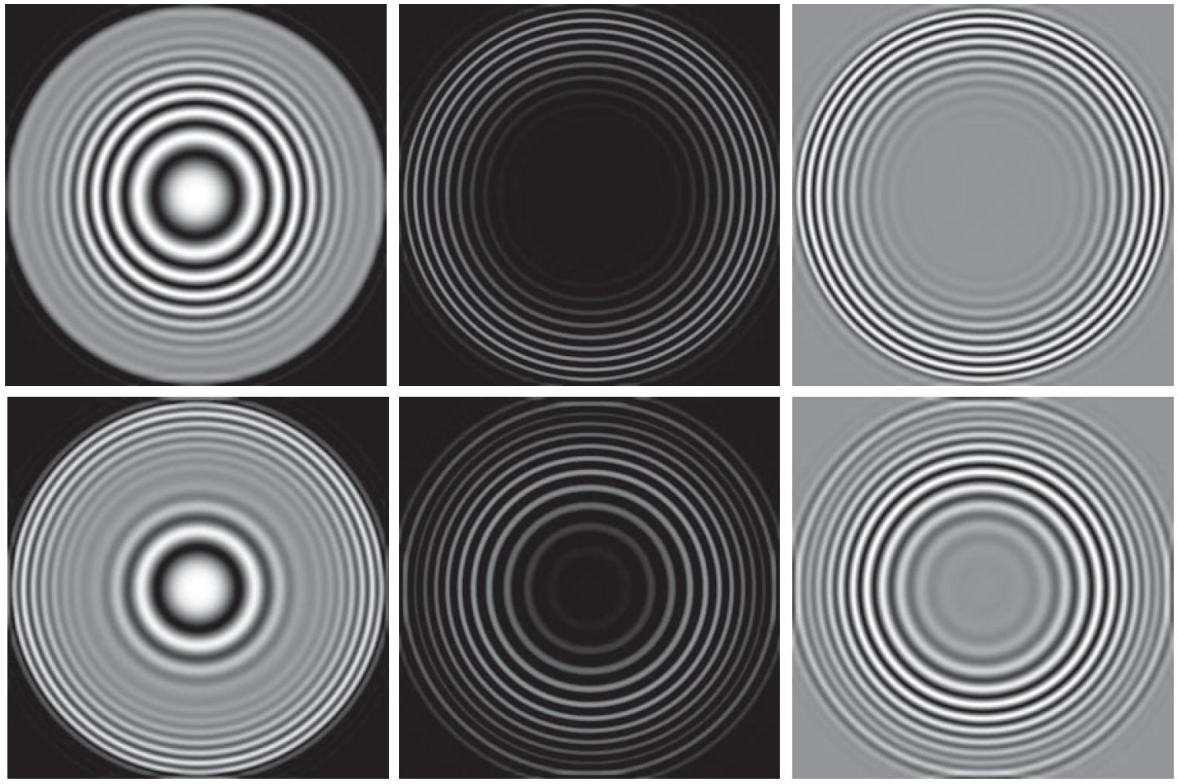
- Representation of these filter in the spatial vs frequency domain.

| Filter Type | Spatial Kernel | Frequency Kernel |
|---|---|---|
| **Lowpass** | $lp(x, y)$ | $LP(u, v)$ |
| **Highpass** | $hp(x, y) = \delta(x, y) - lp(x, y)$ | $HP(u, v) = 1 - LP(u, v)$ |
| **Bandreject** | $br(x, y) = lp_1(x, y) + hp_2(x, y)$ | $BR(u, v) = LP_1(u, v) + HP_2(u, v)$ |
| **Bandpass** | $bp(x, y) = \delta(x, y) - br(x, y)$ | $BP(u, v) = 1 - BR(u, v)$ |



# Highpass, Bandreject, And Bandpass Filters From Lowpass Filters: Example





a b c
d e f

**FIGURE 3.56**
Spatial filtering of the zone plate image. (a) Lowpass result; this is the same as Fig. 3.55(b). (b) Highpass result. (c) Image (b) with intensities scaled. (d) Bandreject result. (e) Bandpass result. (f) Image (e) with intensities scaled.

- More on these in Chapter 4.

# Combining Spatial Enhancement Methods

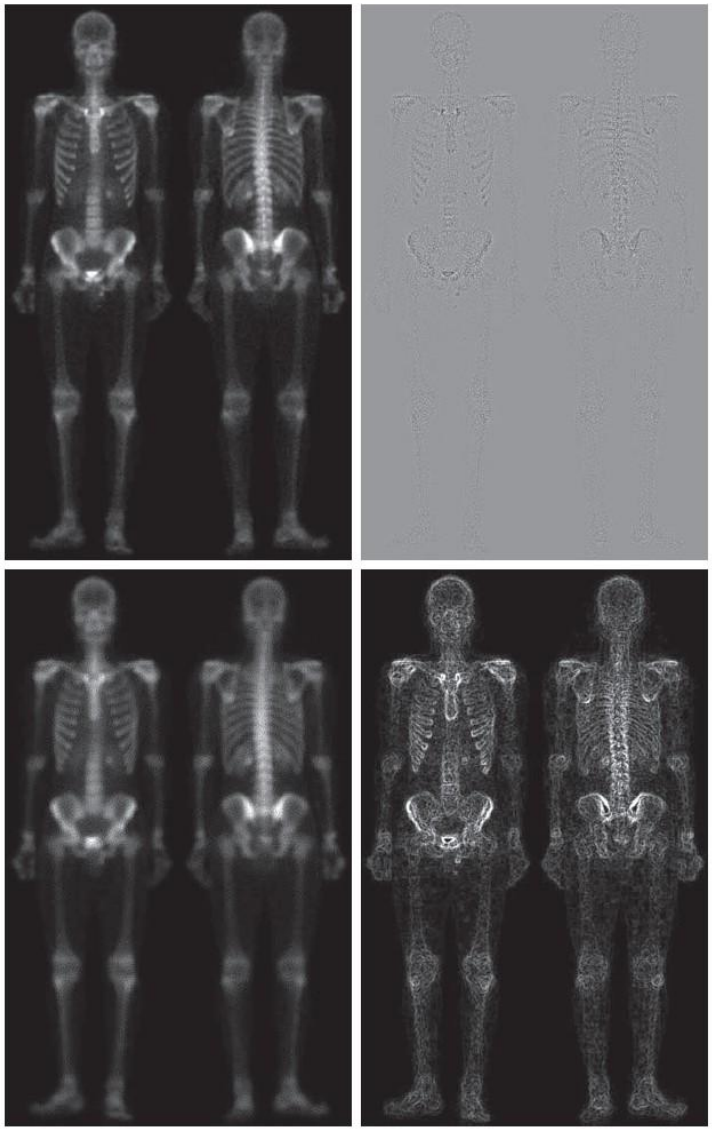
- DIP usually involves application of several complementary techniques to achieve the desired result.
- A nuclear whole body bone scan, which we want to enhance the skeletal details.
- Steps:

  1 Original image

  2 Result of Laplacian operator

  3 Adding the Laplacian image to the original image to get a sharpened (but noisy) image

  4 Result of Sobel operator



# Combining Spatial Enhancement Methods

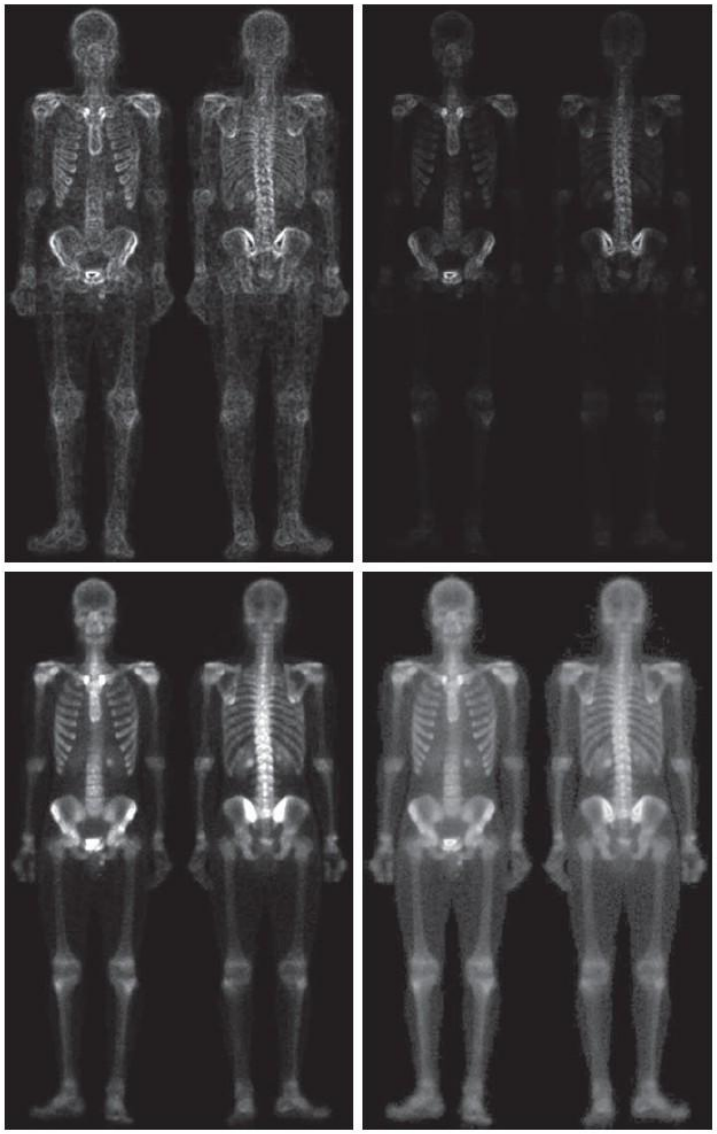
- Steps (cont'd):

  5 Sobel image smoothed by a 5x5 averaging box filter

  6 Mask image by multiplying (5) with (2)

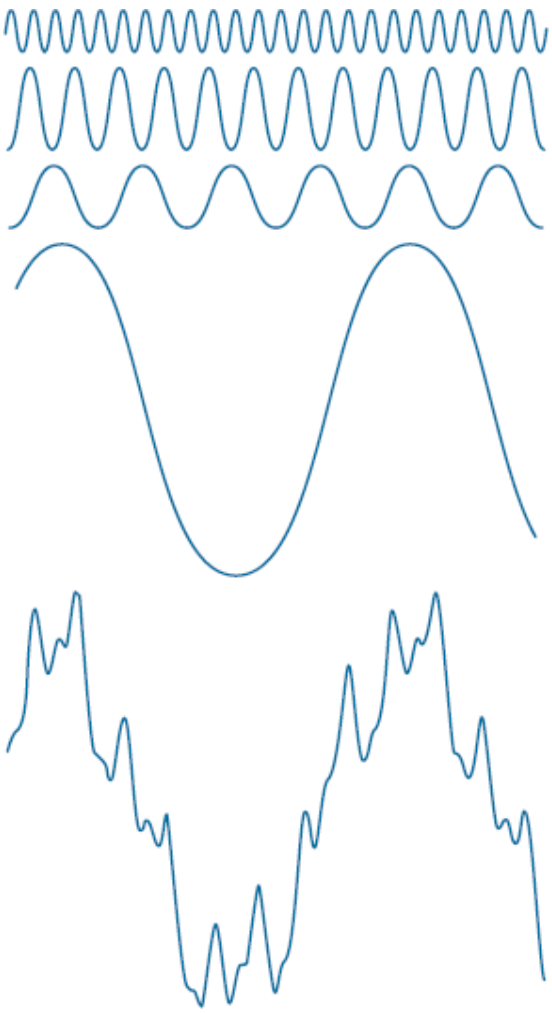
  7 Sharpened image by adding (6) to (1)

  8 Finally, applying a power-law based intensity transformation to enhance the image contrast.



# Introduction to Filtering in the Frequency Domain

- ***Fourier Series***: any periodic function can be represented as the sum of sines and/or cosines of different frequencies, each multiplied by a different coefficient.
- ***Fourier Transform***: functions that are not periodic, but have a finite area under the curve, can be represented as the integral of sines and/or cosines of different frequencies, multiplied by a weighting function.

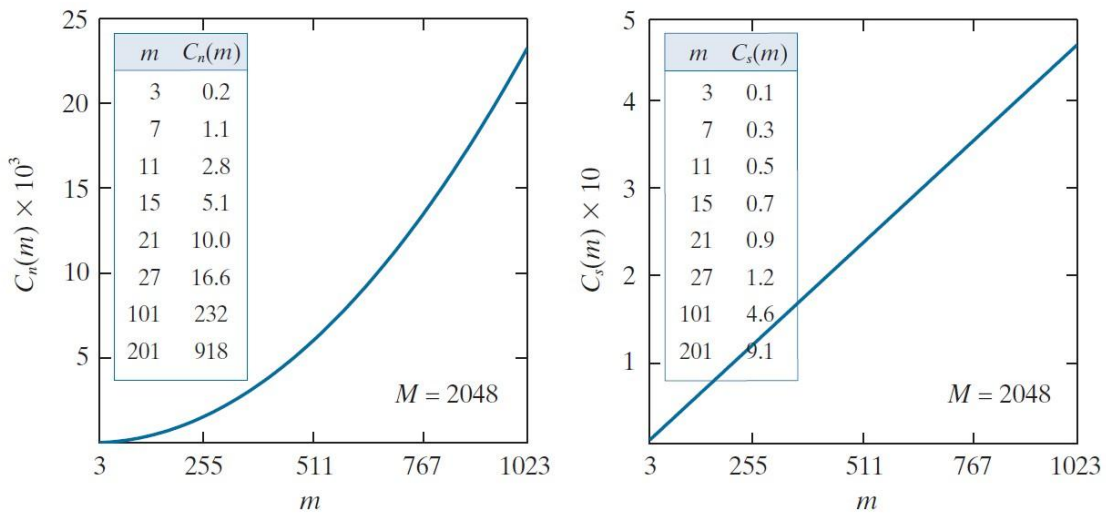


# Introduction to Filtering in the Frequency Domain

- Progress of digital computers and the discovery of Fast Fourier Transform (FFT) algorithms, revolutionized the field greatly.
- For an **MxN** image and an **mxn** kernel, **MNmn** operations is needed.
- If the kernel is separable, then **MN(m+n)** operations are needed.
- If the same filtering is done in a the frequency domain, using FFT, **2MN*log*$_2$MN** operations are needed!

a b

**FIGURE 4.2**
(a) Computational advantage of the FFT over non-separable spatial kernels.
(b) Advantage over separable kernels. The numbers for $C(m)$ in the inset tables are not to be multiplied by the factors of 10 shown for the curves.




| $m$ | $C_n(m)$ |
|---|---|
| 3 | 0.2 |
| 7 | 1.1 |
| 11 | 2.8 |
| 15 | 5.1 |
| 21 | 10.0 |
| 27 | 16.6 |
| 101 | 232 |
| 201 | 918 |

| $m$ | $C_s(m)$ |
|---|---|
| 3 | 0.1 |
| 7 | 0.3 |
| 11 | 0.5 |
| 15 | 0.7 |
| 21 | 0.9 |
| 27 | 1.2 |
| 101 | 4.6 |
| 201 | 9.1 |